Compte rendu Projet serrure électronique

Cécile Janin

- Introduction
- La méthode de travail
  - Organisation des fichiers
  - Simulation du VHDL
- Le module detect
- Le module bascule D
- Le module registre 4 bits
  - Un version simple :
- Le module registre à décalage 4x4bits
- Le module afficheur 7 segments
- Le module comparateur 4 bits
- Le module comparateur 4x4bits
- Le module serrure électronique
- Conclusion

Introduction

Pour ce projet, nous avons dans un premier temps utilisé le logiciel modelsim sous Windows. Cependant, nous avons pris la décision d'utiliser des logiciels en source ouverte natifs linux.

Ainsi, nous avons utilisé :

- OneWareStudio pour l'édition du VHDL

- OneWareStudio et GTKWave pour la visualisation des fichiers de simulation
- GHDL pour la simulation
- Makefile comme script de simulation, lancé depuis le terminal
- Obsidian pour la rédaction du compte rendu
- logisim-evolution pour la rédaction graphique de schémas logique et leur simulation "manuelle"

Si nous souhaitons synthétiser ce projet, il sera possible d'installer quartus sous linux.

Nous détaillerons dans ce document le travail effectué, ainsi que la réponse aux questions aux questions.

La méthode de travail

Organisation des fichiers

Ci-dessous l'organisation des fichiers du projet.

- compte-rendu/ contient les fichiers nécessaire au CR ainsi que le CR en question : images, schéma logisim-évolution, etc.
- simulation/ contient les simulations en .vhd.
- Les fichiers VHDL sont à la racine, ainsi que le Makefile nécessaire à la simulation
- build/ est le dossier de build de GHDL

Simulation du VHDL

Afin de compiler le VHDL de manière automatisée, nous avons utilisé un fichier Makefile, dont le contenu est ci-dessous.

```
# options de compilation
GHDL = ghdl
WORKDIR = build
SIMUDIR = simulation
#GHDL_FLAGS = --std=93c
```

```
# fichiers testbench à compiler
SRC = comparateur_4bits_tb.vhd  comparateur_4x4bits_tb.vhd  registre_decalage_4x4bits_tb.vhd  serrure_elec_tb.vhd
# nom des fichiers de sortie, en .vcd
WAVEFORM = $(addprefix $(SIMUDIR)/, $(SRC:.vhd=.vcd))

all : $(WAVEFORM)

$(SIMUDIR)/%.vcd : %.vhd
    echo "$*"
    ghdl -i --workdir=$(WORKDIR) -P$(WORKDIR) $(GHDL_FLAGS) *.vhd
    ghdl -m --workdir=$(WORKDIR) -P$(WORKDIR) $(GHDL_FLAGS) $*
    ghdl -e --workdir=$(WORKDIR) -P$(WORKDIR) $(GHDL_FLAGS) $*
    ghdl -r --workdir=$(WORKDIR) -P$(WORKDIR) $(GHDL_FLAGS) $* --vcd=$@ $(GHDL_FLAGS)

clean :
    rm -rf $(SIMUDIR)/*
    rm -rf $(WORKDIR)/*
```

Ainsi, la commande make lance la simulation de tous les fichiers listés dans SRC, et make clean efface le contenu des dossiers simulation et build, afin de repartir sur une base saine.

Si on souhaite rajouter un test bench à la simulation, il suffit de rajouter son nom dans la variable SRC.

Il est important de noter qu'une simulation déjà effectuée ne sera pas relancée si le fichier source n'a pas été modifié.

Le module detect

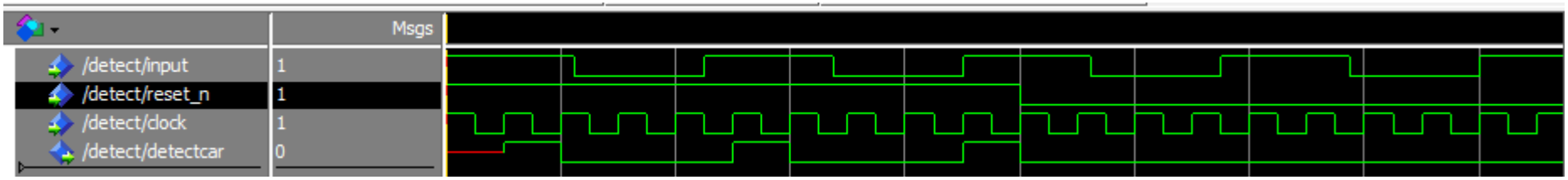
Ci-dessous le code du module :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity detect is
    Port ( input : in STD_LOGIC;
        reset_n : in STD_LOGIC;
        clock : in STD_LOGIC;
        detectcar : out STD_LOGIC);
end entity;

architecture arch of detect is
signal dernier_etat : std_logic := '0';
begin
    process(clock, reset_n)
    begin
        if reset_n = '0' then
            detectcar <= '0';
        elsif rising_edge(clock)
        then
            dernier_etat <= input;
            detectcar <= input and not dernier_etat;
        end if;
    end process;
```

Ci-dessous la simulation manuelle du module detect :



Le module bascule D

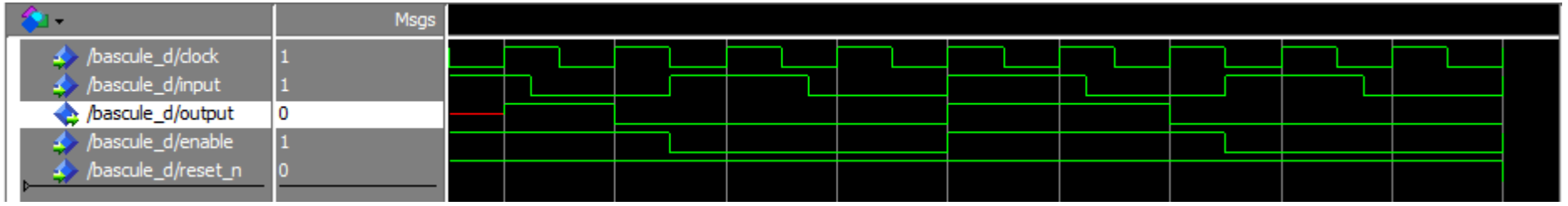
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bascule_d is
    Port ( enable : in STD_LOGIC;
           reset_n : in STD_LOGIC;
           clock : in STD_LOGIC;
           input: in STD_LOGIC;
           output : out STD_LOGIC);
end entity;

architecture arch of bascule_d is
begin
    process(clock, reset_n)
    begin
        if reset_n = '0' then
            output <= '0';
        elsif rising_edge(clock)
        then
            if enable = '1' then
                output <= input;
            end if;
```

```
        end if;
    end process;
```

Ci-dessous la simulation, manuelle :



Le module registre 4 bits

Un version simple :

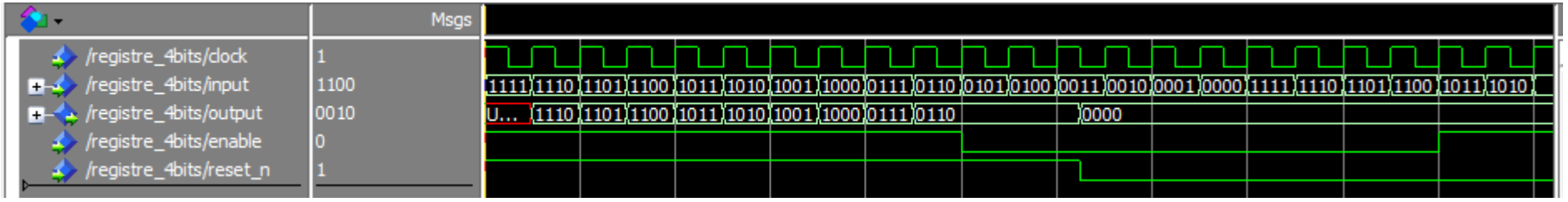
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registre_4bits is
    Port ( enable : in STD_LOGIC;
        reset_n : in STD_LOGIC;
        clock : in STD_LOGIC;
        input: in STD_LOGIC_VECTOR(3 downto 0);
        output : out STD_LOGIC_VECTOR(3 downto 0));
end entity;

architecture arch of registre_4bits is
begin
    process(clock, reset_n)
```

```
    begin
        if reset_n = '0' then
            output <= "0000";
        elsif rising_edge(clock)
        then
            if enable = '1' then
                output <= input;
            end if;
        end if;
    end process;
end architecture;
```

La simulation manuelle associée. On observe bien que le reset remet les bascules D à 0.



Et ci-dessous la version demandée, en utilisant les bascules D. La simulation est strictement similaire à la précédente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registre_4bits is
    Port ( enable : in STD_LOGIC;
        reset_n : in STD_LOGIC;
        clock : in STD_LOGIC;
        input: in STD_LOGIC_VECTOR(3 downto 0);
```

```
        output : out STD_LOGIC_VECTOR(3 downto 0));
end entity;

architecture arch of registre_4bits is
begin
        bascule_d0 : entity work.bascule_d port map
        (
            clock => clock,
            reset_n => reset_n,
            enable => enable,
            input => input(0),
            output => output(0));
        bascule_d1 : entity work.bascule_d port map
        (
            clock => clock,
            reset_n => reset_n,
            enable => enable,
            input => input(1),
            output => output(1)
        );
        bascule_d2 : entity work.bascule_d port map
        (
            clock => clock,
            reset_n => reset_n,
            enable => enable,
            input => input(2),
            output => output(2));
        bascule_d3 : entity work.bascule_d port map
        (
            clock => clock,
```

```
            reset_n => reset_n,
            enable => enable,
            input => input(3),
            output => output(3));
end architecture;
```

Le module registre à décalage 4x4bits

Ci-dessous le code du module:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registre_decalage_4x4bits is
    Port ( enable : in STD_LOGIC;
        reset_n : in STD_LOGIC;
        clock : in STD_LOGIC;
        input: in STD_LOGIC_VECTOR(3 downto 0);
        output_register_0 : out STD_LOGIC_VECTOR(3 downto 0);
        output_register_1 : out STD_LOGIC_VECTOR(3 downto 0);
        output_register_2 : out STD_LOGIC_VECTOR(3 downto 0);
        output_register_3 : out STD_LOGIC_VECTOR(3 downto 0));
end entity;

architecture arch of registre_decalage_4x4bits is
signal internal_0 : std_logic_vector(3 downto 0) := "0000";
signal internal_1 : std_logic_vector(3 downto 0) := "0000";
signal internal_2 : std_logic_vector(3 downto 0) := "0000";
begin
```

```
registre_4bits_0 : entity work.registre_4bits port map
(
    clock => clock,
    reset_n => reset_n,
    enable => enable,
    input => input,
    output => internal_0);

registre_4bits_1: entity work.registre_4bits port map
(
    clock => clock,
    reset_n => reset_n,
    enable => enable,
    input => internal_0,
    output => internal_1);

registre_4bits_2 : entity work.registre_4bits port map
(
    clock => clock,
    reset_n => reset_n,
    enable => enable,
    input => internal_1,
    output => internal_2);

registre_4bits_3 : entity work.registre_4bits port map
(
    clock => clock,
    reset_n => reset_n,
    enable => enable,
    input => internal_2,
```

```
            output => output_register_3);

        output_register_0 <= internal_0;
        output_register_1 <= internal_1;
        output_register_2 <= internal_2;
end architecture;
```

Ainsi que celui du test bench. On remarquera le signal finished qui désactive le process clock à la fin de la simulaiton.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity registre_decalage_4x4bits_tb is
end entity registre_decalage_4x4bits_tb;

architecture arch of registre_decalage_4x4bits_tb is
    constant periode : time := 10 ns;
    signal enable_tb : std_logic;
    signal clock_tb : std_logic := '0';
    signal reset_n_tb : std_logic;
    signal input_tb : std_logic_vector(3 downto 0);
    signal output_0_tb : std_logic_vector(3 downto 0);
    signal output_1_tb : std_logic_vector(3 downto 0);
    signal output_2_tb : std_logic_vector(3 downto 0);
    signal output_3_tb : std_logic_vector(3 downto 0);

    signal finished : std_logic := '0';
```

```
begin

    -- ce processus limite le temps de simulation
    sim_time_proc: process
    begin
        wait for 1 us;
        finished <= '1';
        wait;
    end process sim_time_proc;

    clock : process
    begin
        while finished /= '1' loop
            clock_tb <= not clock_tb;
            wait for periode/2;
        end loop;
        wait;
    end process clock;

    test: process
    begin
        input_tb <= "1010"; -- a en premier
        reset_n_tb <= '1';
        enable_tb <= '0';


        for j in 0 to 5 loop
            if j = 5 then
                enable_tb <= '1';
            end if;
```

```
        wait for 11 ns;
        enable_tb <= '0';
    end loop;

    input_tb <= "1110";

    for j in 0 to 5 loop
        if j = 5 then
            enable_tb <= '1';
        end if;
        wait for 10 ns;
        enable_tb <= '0';
    end loop;

    input_tb <= "0001";

    for j in 0 to 5 loop
        if j = 5 then
            enable_tb <= '1';
        end if;
        wait for 10 ns;
        enable_tb <= '0';
    end loop;

    input_tb <= "1011";

    for j in 0 to 5 loop
        if j = 5 then
            enable_tb <= '1';
        end if;
```

```
            wait for 10 ns;
            enable_tb <= '0';
        end loop;

        wait for 10 ns;
        reset_n_tb <= '0';

        wait for 10 ns;
        reset_n_tb <= '1';

        for j in 0 to 5 loop
            if j = 5 then
                enable_tb <= '1';
            end if;
            wait for 10 ns;
            enable_tb <= '0';
        end loop;

        report "End of test. Verify that no error was reported.";

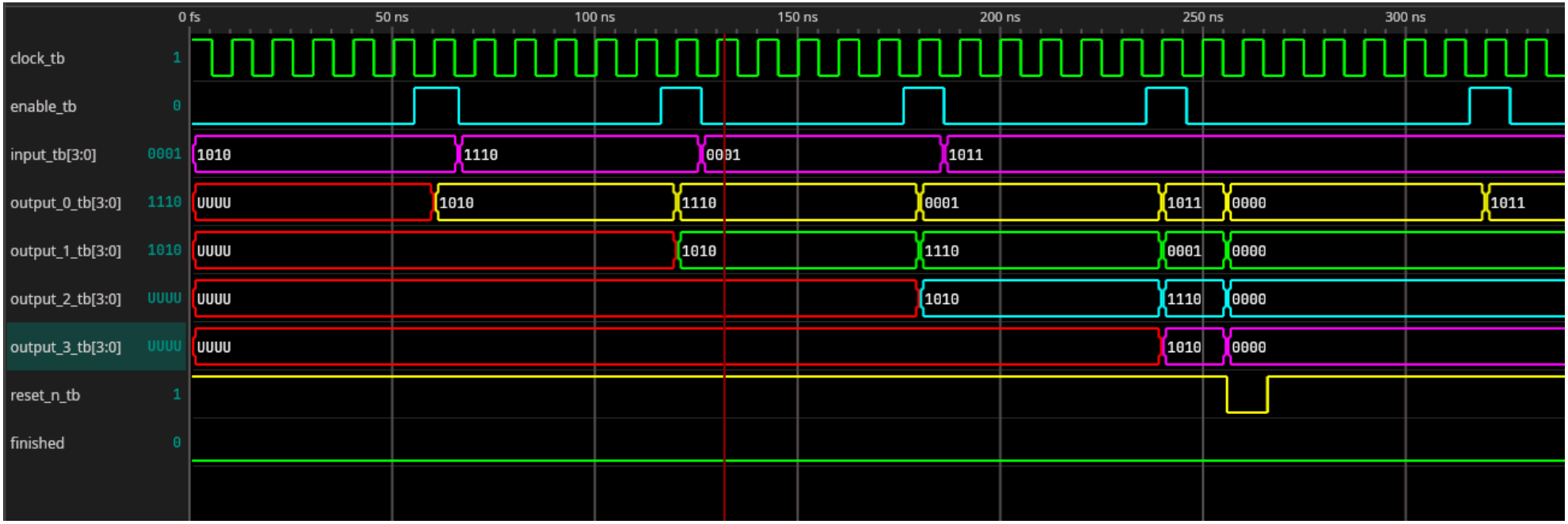
        wait;
    end process test;


    UUT : entity work.registre_decalage_4x4bits
    port map
    (
        enable => enable_tb,
        clock => clock_tb,
        reset_n => reset_n_tb,
```

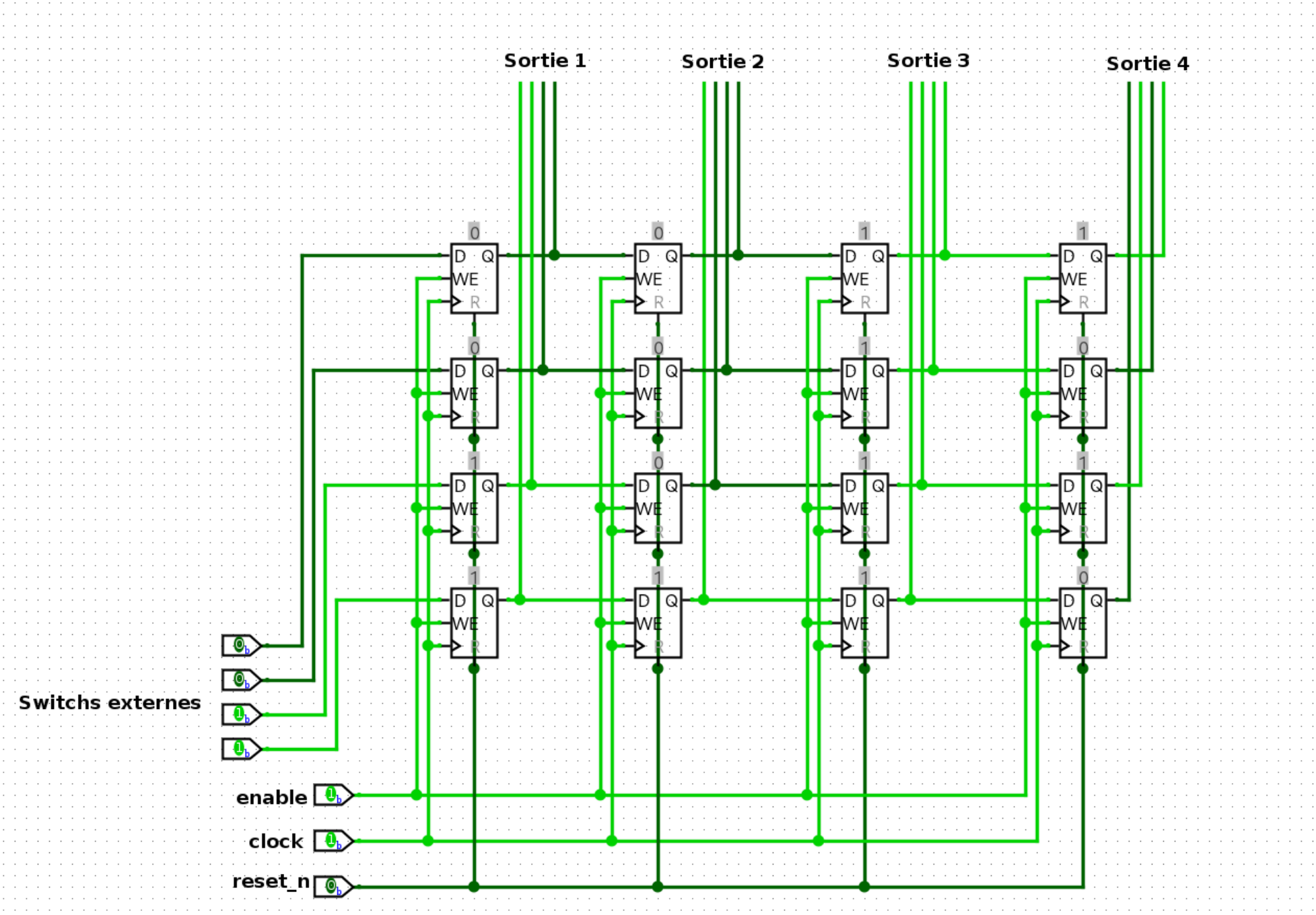
```
        input => input_tb,
        output_register_0 => output_0_tb,
        output_register_1 => output_1_tb,
        output_register_2 => output_2_tb,
        output_register_3 => output_3_tb);

end architecture arch;
```

Simulation par le test bench, sous OneWareStudio. On remarque bien que les valeurs entrées se décalent, puis sont mis à 0 lors du reset.



Le schéma ci-dessous représente le registre 4x4bits.



Le module afficheur 7 segments

Ci-dessous le code :

```
library ieee;
use ieee.std_logic_1164.all;

entity decodeur_7_segments is
    port (
        input: in STD_LOGIC_VECTOR(3 downto 0);
        afficheur_7_segments : out STD_LOGIC_VECTOR(6 downto 0) := "0000000"
    );
end entity;

architecture arch of decodeur_7_segments is
begin
    process (input)
    begin
        case (input) is
            when x"0" => afficheur_7_segments <= "1111110";
            when x"1" => afficheur_7_segments <= "0110000";
            when x"2" => afficheur_7_segments <= "1101101";
            when x"3" => afficheur_7_segments <= "1111001";
            when x"4" => afficheur_7_segments <= "0110011";
            when x"5" => afficheur_7_segments <= "1011011";
            when x"6" => afficheur_7_segments <= "1011111";
            when x"7" => afficheur_7_segments <= "1110000";
            when x"8" => afficheur_7_segments <= "1111111";
            when x"9" => afficheur_7_segments <= "1111011";
```

```
            when others => afficheur_7_segments <= "0000000";
        end case;
    end process;
end architecture;
```

Puis le code du décodeur 4x :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity decodeur_7_segments_x4 is
    Port (
        input0: in STD_LOGIC_VECTOR(3 downto 0);
        input1: in STD_LOGIC_VECTOR(3 downto 0);
        input2: in STD_LOGIC_VECTOR(3 downto 0);
        input3: in STD_LOGIC_VECTOR(3 downto 0);
        afficheur_7_segments_0 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000";
        afficheur_7_segments_1 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000";
        afficheur_7_segments_2 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000";
        afficheur_7_segments_3 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000");
end entity;

architecture arch of decodeur_7_segments_x4 is
begin
    afficheur_0 : entity work.decodeur_7_segments port map
    (
        input => input0,
        afficheur_7_segments => afficheur_7_segments_0);
```

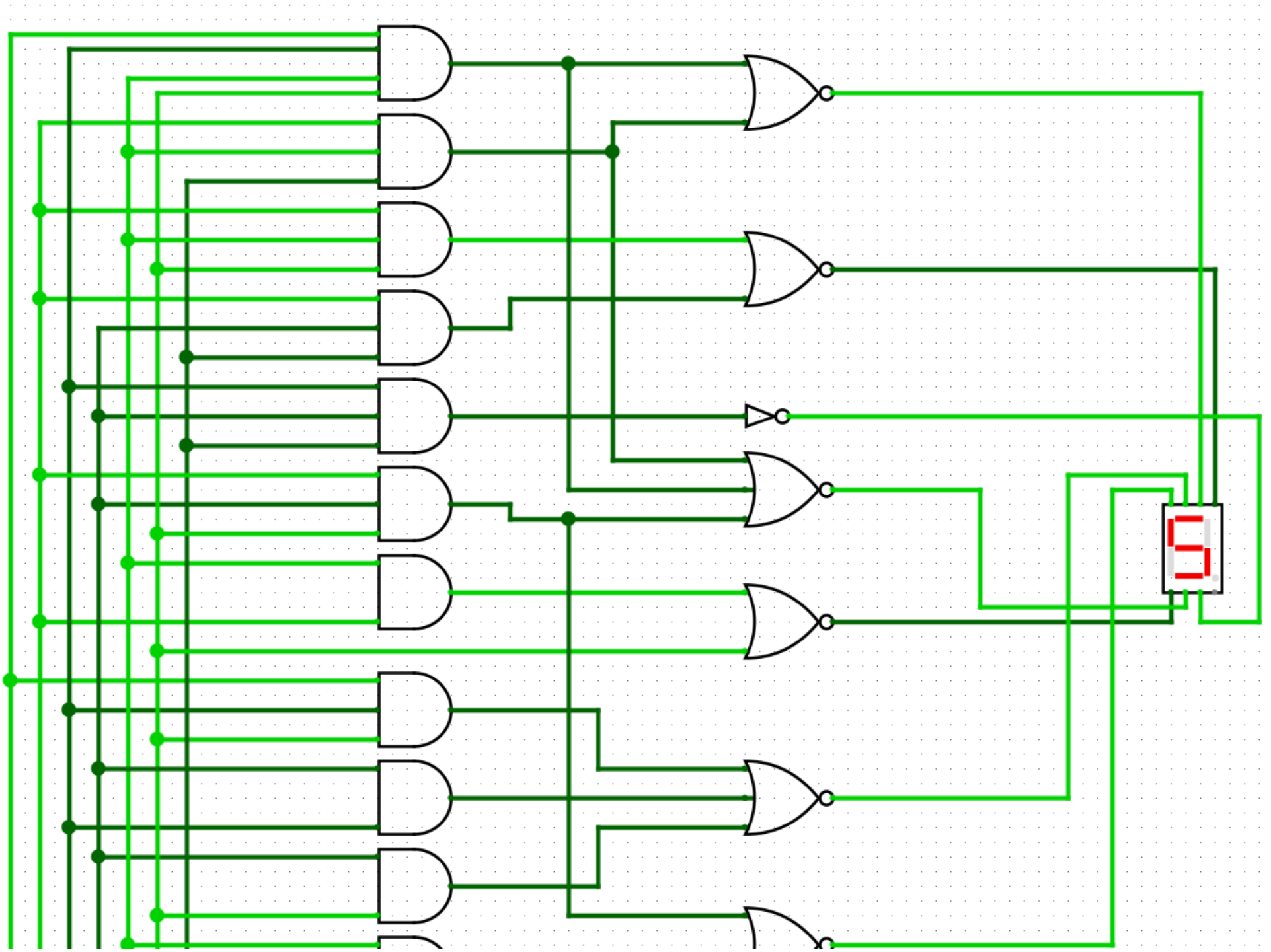
```
    afficheur_1: entity work.decodeur_7_segments port map
    (
        input => input1,
        afficheur_7_segments => afficheur_7_segments_1);

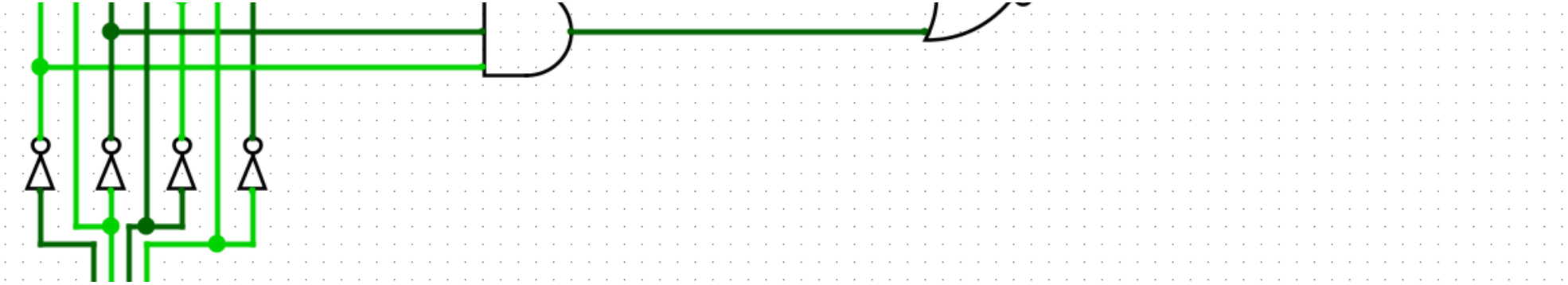
    afficheur_2 : entity work.decodeur_7_segments port map
    (
        input => input2,
        afficheur_7_segments => afficheur_7_segments_2);

    afficheur_3 : entity work.decodeur_7_segments port map
    (
        input => input3,
        afficheur_7_segments => afficheur_7_segments_3);

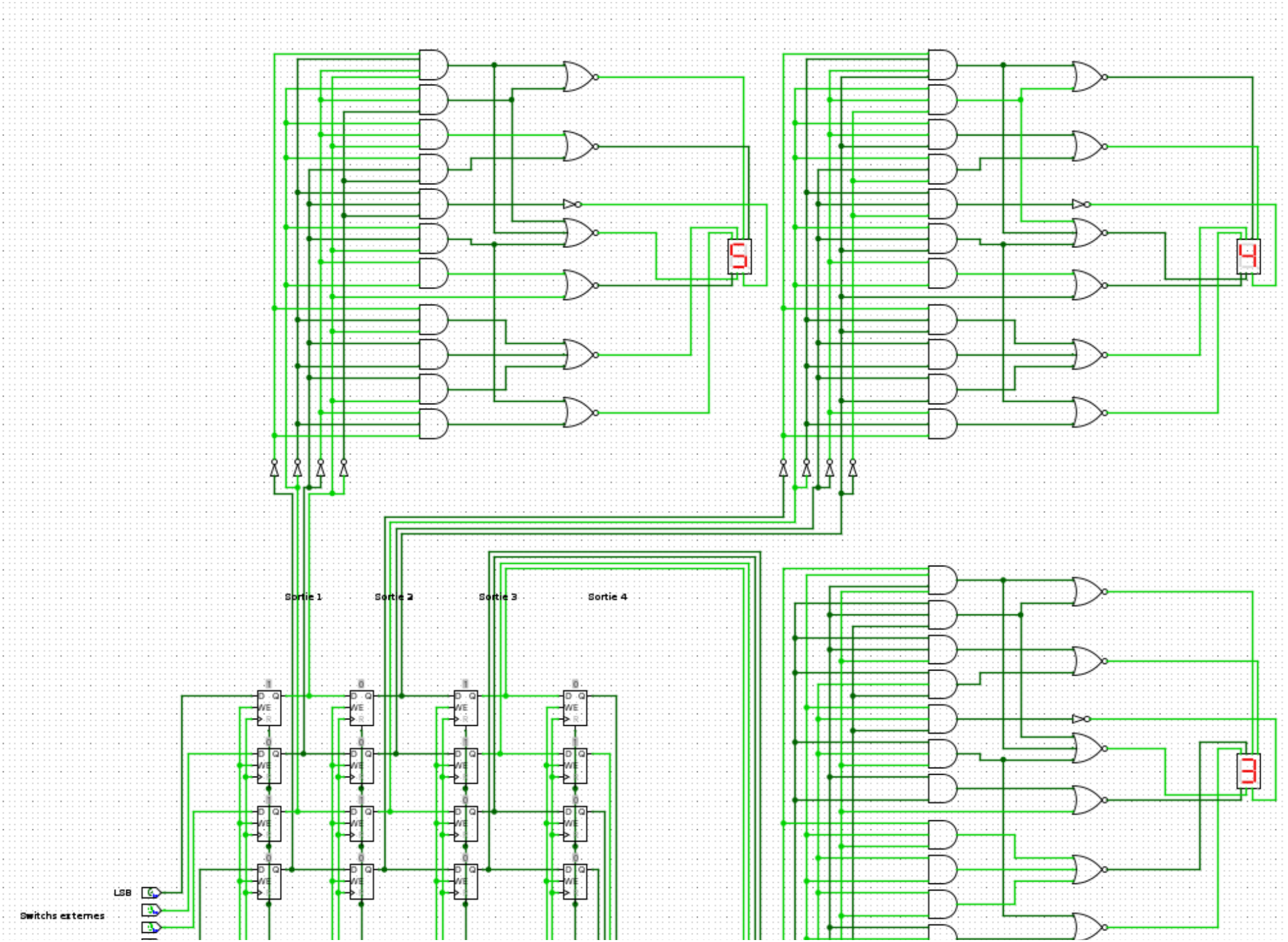
end architecture;
```

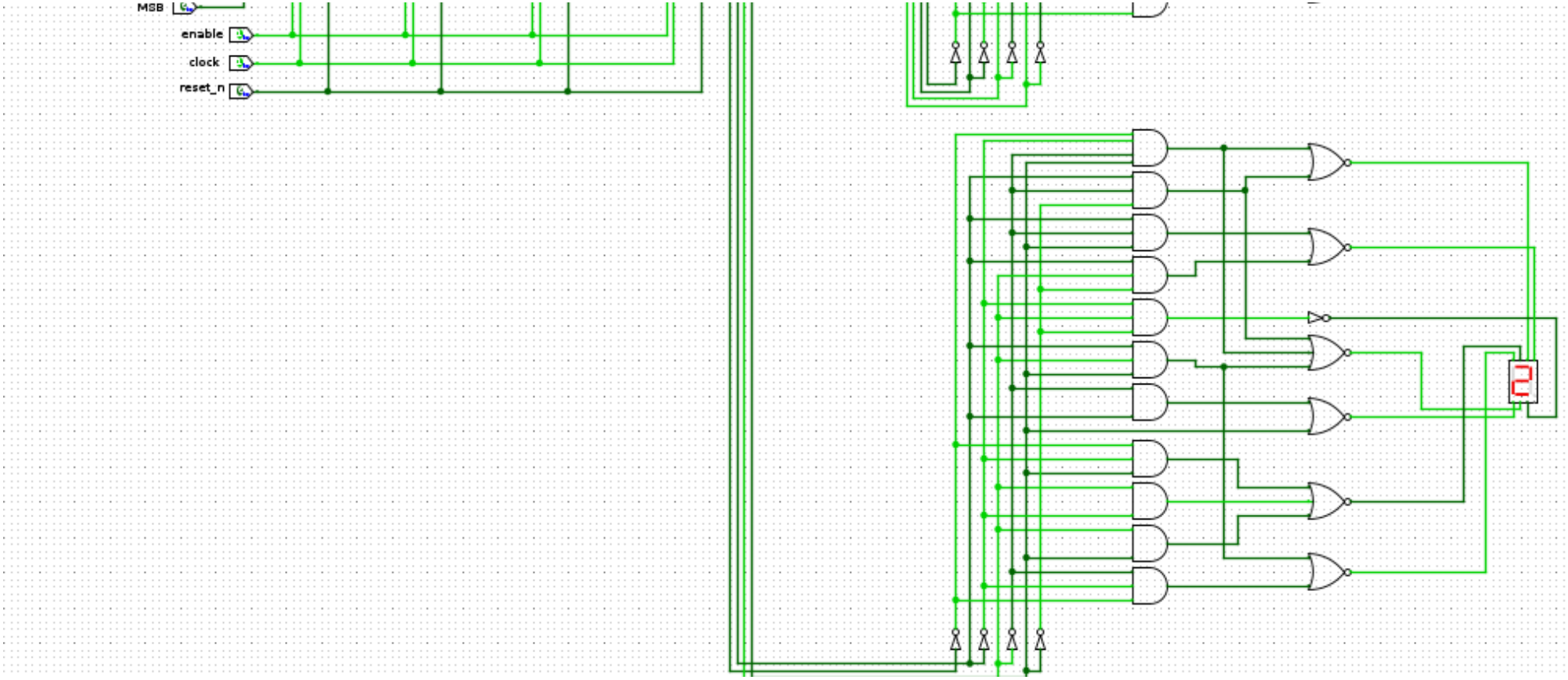
Le schéma d'un seul décodeur :





Et enfin le schéma global. On remarque bien que pour la suite de chiffre 2,3,4,5, les afficheurs se décalent les uns après les autres.


Sortie 1
Sortie 2
Sortie 3
Sortie 4
LSB
Switchs externes



Le module comparateur 4 bits

Ci-dessous le code du comparateur :

```
library ieee;
use ieee.std_logic_1164.all;

entity comparateur_4bits is
    port (
        input : in std_logic_vector(3 downto 0);
        code : in std_logic_vector(3 downto 0);
```

```
        output : out std_logic := '0'
    );
end entity;

architecture arch of comparateur_4bits is

begin
    output <= '1' when input = code else '0';
end architecture;
```

Ainsi que son test bench associé :

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity comparateur_4bits_tb is
end entity;

architecture arch of comparateur_4bits_tb is
    constant periode : time := 10 ns;

    signal input_tb : std_logic_vector(3 downto 0) := "0000";
    signal code_tb : std_logic_vector(3 downto 0):= "0000";
    signal output_tb : std_logic := '0';

    signal finished : std_logic := '0';
begin
```

```
-- this process define the simulation time
sim_time_proc: process
begin
    wait for 110 ns;
    finished <= '1';
    wait;
end process sim_time_proc;


test: process
begin
    code_tb <= "1010"; -- le code
    wait for 10 ns;

    for i in 0 to 9 loop
        input_tb <= std_logic_vector( signed(input_tb) + 1 );
        wait for 10 ns;
    end loop;

    report "End of test. Please verify that no error was reported.";

    wait;
end process test;


UUT : entity work.comparateur_4bits
port map
(
    input => input_tb,
    output => output_tb,
```

```
        code => code_tb
    );

end architecture arch;
```

Ci-dessous la simulation. On remarque bien que la sortie ne s'active que si la bonne valeur est donnée.



Question : Non car il s'agit d'un module en logique combinatoire et non séquencielle.

Le module comparateur 4x4bits

Le code du comparateur :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity comparateur_4x4bits is
    Port ( input : in std_logic_vector(15 downto 0);
        code : in std_logic_vector(15 downto 0);
        output : out std_logic := '0');
end entity;
```

```
architecture arch of comparateur_4x4bits is
    signal output_x4 : std_logic_vector(3 downto 0) := "0000";
begin
    registre_4bits_0 : entity work.comparateur_4bits port map
    (
        input => input(3 downto 0),
        code => code(3 downto 0),
        output => output_x4(0));

    registre_4bits_1: entity work.comparateur_4bits port map
    (
        input => input(7 downto 4),
        code => code(7 downto 4),
        output => output_x4(1));

    registre_4bits_2 : entity work.comparateur_4bits port map
    (
        input => input(11 downto 8),
        code => code(11 downto 8),
        output => output_x4(2));

    registre_4bits_3 : entity work.comparateur_4bits port map
    (
        input => input(15 downto 12),
        code => code(15 downto 12),
        output => output_x4(3));

    output <= '1' when output_x4 = "1111" else '0';
end architecture;
```

Ci-dessous le code du test bench.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity comparateur_4x4bits_tb is
end entity;

architecture arch of comparateur_4x4bits_tb is
    signal input_tb : std_logic_vector(15 downto 0) := "0000000000000000";
    signal code_tb : std_logic_vector(15 downto 0):= "0000000000000000";
    signal output_tb : std_logic := '0';

    signal finished : std_logic := '0';
begin

    -- this process define the simulation time
    sim_time_proc: process
    begin
        wait for 660 us;
        finished <= '1';
        wait;
    end process sim_time_proc;


    test: process
    begin
        code_tb <= x"beef"; -- le code
```

```
        wait for 10 ns;

        for i in 0 to 65535 loop
            input_tb <= std_logic_vector( signed(input_tb) + 1 );
            wait for 10 ns;
        end loop;

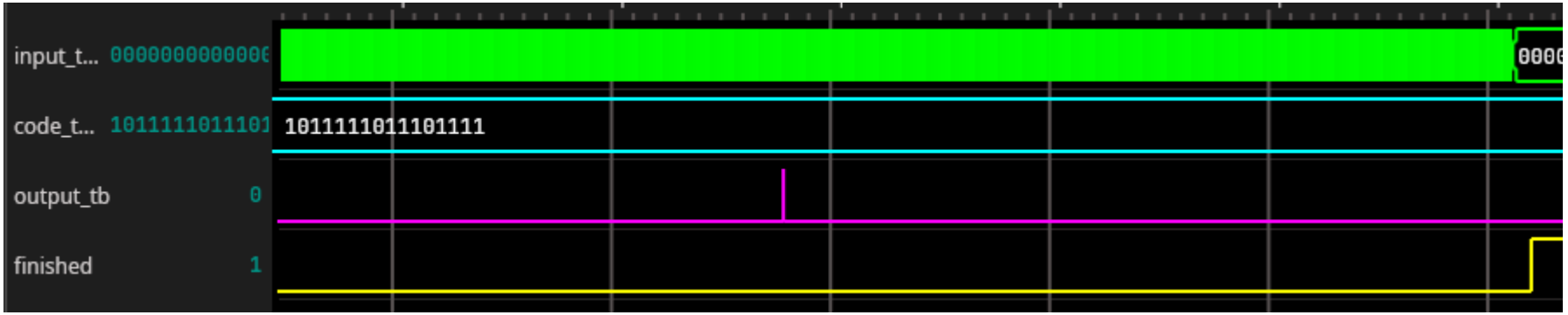
        report "End of test. Please verify that no error was reported.";

        wait;
    end process test;

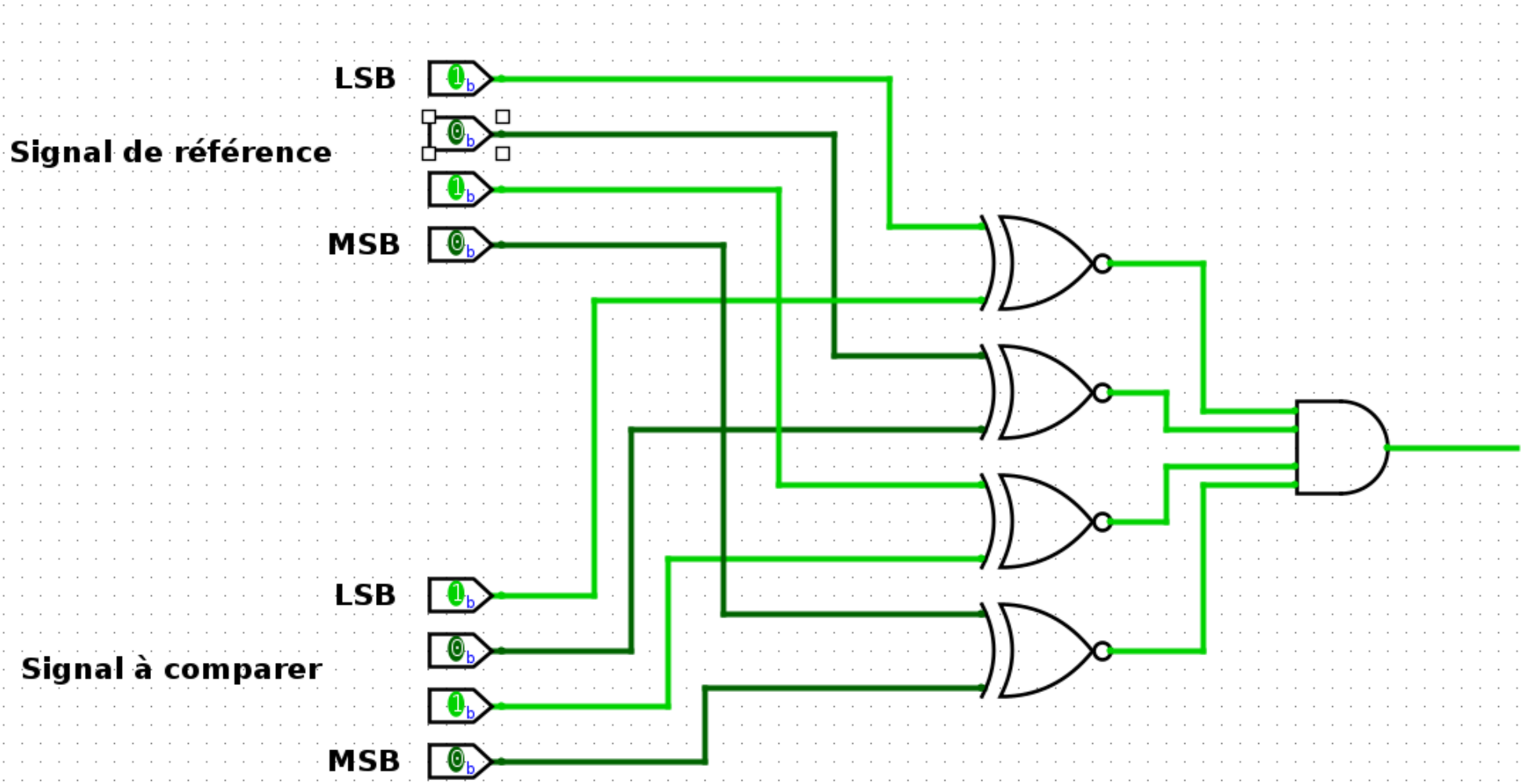

    UUT : entity work.comparateur_4x4bits
    port map
    (
        input => input_tb,
        output => output_tb,
        code => code_tb
    );

end architecture arch;
```

Le résultat de simulation est ci-dessous. Toutes les valeurs ont été testées, on remarque bien que la sortie est à l'état haut que pour une seule d'entre elle : celle donnée en entrée.



Ci-dessous le schéma pour un comparateur 4bits. Un 16 bits reprends le même principe.



Question : Pour que toutes les combinaisons soient vraies, imposez simplement un état 1 à a sortie du comparateur.

Le module serrure électronique

Ci-dessous le code du module :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity serrure_electronique is
    Port (
        input_interrupteurs : in std_logic_vector(3 downto 0);
        input_bouton : in std_logic;
        afficheur_7_segments_0 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000";
        afficheur_7_segments_1 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000";
        afficheur_7_segments_2 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000";
        afficheur_7_segments_3 : out STD_LOGIC_VECTOR(6 downto 0) := "0000000";
        reset_n : in std_logic;
        clock : in std_logic;
        ouverture : out std_logic:= '0'
    );
end entity;

architecture arch of serrure_electronique is
    signal enable_serrure : std_logic;
    signal code_complet : std_logic_vector(15 downto 0) := x"1234";
    signal input_afficheurs_7_segments_x4 : std_logic_vector(15 downto 0);
begin
    registre_decalage_4x4bits : entity work.registre_decalage_4x4bits port map
    (
        output_register_0 => input_afficheurs_7_segments_x4(3 downto 0),
        output_register_1 => input_afficheurs_7_segments_x4(7 downto 4),
        output_register_2 => input_afficheurs_7_segments_x4(11 downto 8),
        output_register_3 => input_afficheurs_7_segments_x4(15 downto 12),
        input => input_interrupteurs,
```

```
        reset_n => reset_n,
        clock => clock,
        enable => enable_serrure

    );

    decodeur_7_segments_x4: entity work.decodeur_7_segments_x4 port map
    (
        input0 => input_afficheurs_7_segments_x4(3 downto 0),
        input1 => input_afficheurs_7_segments_x4(7 downto 4),
        input2 => input_afficheurs_7_segments_x4(11 downto 8),
        input3 => input_afficheurs_7_segments_x4(15 downto 12),
        --besoin de les spécifier, ou on peux les relier directement à des broches ?
        -- la je spécifie :
        afficheur_7_segments_0 => afficheur_7_segments_0,
        afficheur_7_segments_1 => afficheur_7_segments_1,
        afficheur_7_segments_2 => afficheur_7_segments_2,
        afficheur_7_segments_3 => afficheur_7_segments_3
    );

    detect : entity work.detect port map
    (
        reset_n => reset_n,
        clock => clock,
        input => input_bouton,
        detectcar => enable_serrure
    );

    comparateur_4x4bits : entity work.comparateur_4x4bits port map
    (
```

```
        input => input_afficheurs_7_segments_x4,
        code => code_complet,
        output => ouverture
    );

end architecture;
```

Et enfin le code du test bench :

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity serrure_elec_tb is
end entity;

architecture arch of serrure_elec_tb is
    constant periode : time := 1 ns;

    signal clock_tb : std_logic := '0';
    signal reset_n_tb : std_logic := '1';
    signal ouverture_tb : std_logic;

    signal input_bouton_tb : std_logic := '0';
    signal input_interrupteurs_tb : std_logic_vector(3 downto 0) := x"0";

    signal afficheur_7_segments_0_tb : STD_LOGIC_VECTOR(6 downto 0) := "0000000";
    signal afficheur_7_segments_1_tb : STD_LOGIC_VECTOR(6 downto 0) := "0000000";
    signal afficheur_7_segments_2_tb : STD_LOGIC_VECTOR(6 downto 0) := "0000000";
```

```
    signal afficheur_7_segments_3_tb : STD_LOGIC_VECTOR(6 downto 0) := "0000000";

    signal finished : std_logic := '0';
begin

    -- ce processus limite le temps de simulation
    sim_time_proc: process
    begin
        wait for 130 ns;
        finished <= '1';
        wait;
    end process sim_time_proc;

    clock : process
    begin
        while finished /= '1' loop
            clock_tb <= not clock_tb;
            wait for periode/2;
        end loop;
        wait;
    end process clock;


    test: process
    begin

        input_interrupteurs_tb <= "0001";

        wait for 10 ns;
        input_bouton_tb <= '1';
```

```
        wait for 10 ns;
        input_bouton_tb <= '0';

        input_interrupteurs_tb <= "0010";

        wait for 10 ns;
        input_bouton_tb <= '1';
        wait for 10 ns;
        input_bouton_tb <= '0';

        input_interrupteurs_tb <= "0011";

        wait for 10 ns;
        input_bouton_tb <= '1';
        wait for 10 ns;
        input_bouton_tb <= '0';

        input_interrupteurs_tb <= "0100";

        wait for 10 ns;
        input_bouton_tb <= '1';
        wait for 10 ns;
        input_bouton_tb <= '0';

        wait for 10 ns;
        reset_n_tb <= '0';
        wait for 10 ns;
        reset_n_tb <= '1';

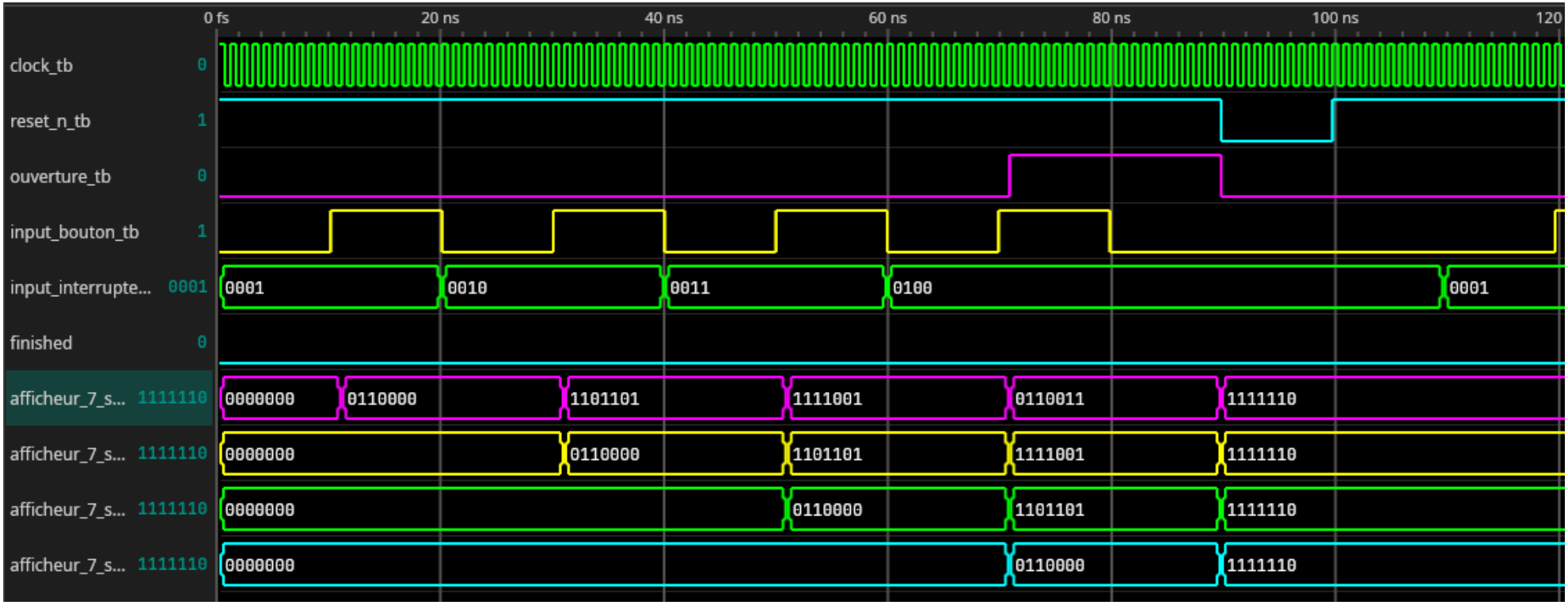
        wait for 10 ns;
```

```
        report "End of test. Please verify that no error was reported.";
        wait;
    end process test;

    UUT : entity work.serrure_electronique
    port map
    (
        input_interrupteurs => input_interrupteurs_tb,
        input_bouton => input_bouton_tb,
        afficheur_7_segments_0 => afficheur_7_segments_0_tb,
        afficheur_7_segments_1 => afficheur_7_segments_1_tb,
        afficheur_7_segments_2 => afficheur_7_segments_2_tb,
        afficheur_7_segments_3 => afficheur_7_segments_3_tb,
        reset_n => reset_n_tb,
        clock => clock_tb,
        ouverture => ouverture_tb
    );

end architecture arch;
```

Ci-dessous la simulation. On remarque bien que tant que nous n'avons pas la bonne combinaison dans le registre 4x4bits, la sortie n'est pas à 1.



Conclusion

Durant ce TP, nous avons conçu un code VHDL pour un serrure électronique à 4 digits. Cela nous a permis, dans un premier temps d'éprouver modelsim, puis de nous approprier des outils open source compatibles linux.

Nous avons bien réussi à faire fonctionner la serrure électronique.